

What you actually need to learn

For an artist who became a vibecoder building Colourmap. No CS degree, big dreams, possibly heading into a master's at Politecnico Milano. This is your minimum-viable map of what to learn — when, why, and how deep.

*The honest truth: **you don't need to learn everything**. You're not training to be a senior engineer at Google. You're training to ship one specific app — Colourmap — and to be the kind of artist-builder who can hold their own in a room of designers, engineers, and musicians. That's a much narrower (and more interesting) target than "learn to code."*

How to read this

10 modules. Each has:

- What it is, in one paragraph an artist would understand
- Why **you specifically** need it for Colourmap
- What to learn first (the 20% that gets you 80% of the way)
- Resources I'd actually recommend (not a list of every YouTube tutorial)
- An honest "you can ignore this if..." escape hatch

Priority labels: **NOW** = needed this month · **SOON** = next 3 months · **LATER** = once Colourmap has 1k users · **OPTIONAL** = nice-to-have.

. . .

01 *The stack you're already using* **NOW**

You're shipping in **Next.js + React + TypeScript + Tailwind + Bun + Vercel + Supabase**. Most of these you've been using without fully understanding. That's fine for shipping — it's not fine forever. Each one is a tool with sharp edges.

What to learn first (in this order)

1. **JavaScript fundamentals** — closures, promises, async/await, the event loop.
Without these, every other thing on this list is a mystery.
2. **TypeScript types** — interfaces, generics, narrowing. You'll feel a 10× productivity gain when you stop fighting the types and start using them as scaffolding.
3. **React hooks** — `useState`, `useEffect`, `useRef`, `useCallback`. The trap is *useEffect*: 80% of React bugs come from misusing it.
4. **Next.js App Router basics** — server components vs. client components, when to use which, layouts, route groups.
5. **Tailwind utility classes** — you mostly know this from doing. Read the docs once front-to-back; you'll find shortcuts.

Best resources

- **Josh Comeau's React for the rest of us** (joshwcomeau.com) — best beginner-but-not-dumb React content on the web.
- **TypeScript Handbook** (typescriptlang.org/docs) — actually written for humans.
- **Next.js docs** (nextjs.org/docs) — App Router section is what you need.

You can ignore deep React internals (fiber, reconciler) until you're optimising for performance. You will *not* need to know how Webpack works. You will *not* need to read the Next.js source. Stay shallow until something breaks.

02 Web Audio — your differentiator **NOW**

The Chill Machine, Groove Machine, Magic Maker, Lo-fi Looper — all built on the **Web Audio API**. This is your edge over every other "wellness app." Most developers never touch this; you're already using it.

What to learn first

1. **The audio graph model** — sources → nodes → destination. Everything is a node. Once this clicks, everything else is reading docs.
2. **Oscillators + envelopes** — sine / square / triangle / sawtooth + ADSR. This is sound design at its most basic. Master this and you can recreate most synths.
3. **Buffer sources + sample playback** — how the real-instrument samples work in your code.
4. **Filters + effects** — biquad filters (lowpass, highpass, bandpass), delay, convolution reverb. Each unlocks a category of sound.
5. **The scheduler pattern** — your `scheduleNextNotes()` function in GrooveMachine. The 100ms-lookahead trick is foundational; once you get it, you can build any rhythm-based tool.

Best resources

- **MDN Web Audio API docs** — surprisingly well-written.
- **Tonejs.github.io** source code — even if you don't use Tone.js, reading its source teaches you idiomatic Web Audio.
- **"A Tale of Two Clocks"** by Chris Wilson — the article that explains why the scheduler pattern works.

This is the area where you can *actually become world-class* over 2–3 years. Most music-app builders are former DSP engineers, not artists. You have the artist's ear. Keep building, keep listening to references, keep tweaking. The combination is rare.

Web Audio is the *tool*; this is what to *do with it*. As a musician you already have most of this — but the formal vocabulary (intervals, voice leading, timbre theory) lets you reason about what you're hearing.

What to learn first

1. **Intervals + scales** — pentatonic, major, minor, modes (dorian, mixolydian, phrygian). You're already using these in code.
2. **Chord progressions** — I-V-vi-IV, ii-V-I, modal interchange. Your Groove presets need these.
3. **Frequency vs. pitch** — $A_4 = 440$ Hz, octave doubles. The `baseNote * 2 ** (semitones/12)` formula in your code lives because of this.
4. **Envelope shapes** — pluck (fast attack, fast decay) vs. pad (slow attack, slow release) vs. percussion (instant attack, almost no sustain). This is what makes a "sound" feel like an instrument.
5. **Mixing fundamentals** — EQ (cut more than boost), compression, reverb send vs. insert, stereo width.

Best resources

- **Andrew Huang YouTube** — accessible production basics.
- **"How Music Works"** — David Byrne. Not technical but reframes how you think about songs as *contexts*.
- **Adam Neely YouTube** — for music theory delivered with a straight face but real depth.

Colourmap will eventually need to store things across devices — check-ins, projects, Circle missions, saved sounds. Today this works in `localStorage`; tomorrow you need real persistence with auth.

What to learn first

1. **SQL fundamentals** — `SELECT`, `INSERT`, `UPDATE`, `JOIN`, foreign keys. Most of "backend" is just SQL with extra steps.
2. **Schema design** — why you put email on user, not on post. Why you don't store JSON blobs when columns would do. Normalisation.
3. **Migrations** — how schemas evolve over time without breaking production. Drizzle (which you're using) makes this easy.
4. **Row-level security (RLS)** in Supabase — the security model that lets the database itself enforce who can read what.
5. **RESTful API design** — why you have `GET /circles/:id` and `POST /circles/:id/missions`, not `POST /api/getCircle`.

Best resources

- **Supabase docs** — they're really good. Read the auth + RLS sections.
- **"SQL for the Lazy"** — Mode Analytics tutorial (mode.com).
- **Drizzle ORM docs** — for the migration patterns you're using.

You probably won't need to know about indexing, query optimisation, or stored procedures until 10k+ users. Stay shallow.

This is the area artists most reliably skip and most reliably regret. Even a small intentional app needs to handle passwords, sessions, and not leak data correctly.

What to learn first

1. **Sessions vs. JWTs** — what your auth cookie actually is, why it works, how it can be stolen.
2. **HTTPS + CORS** — why every API call needs to be on the right domain with the right headers.
3. **OWASP Top 10** — the 10 most common security mistakes. Read once, reread once a year.
4. **Password handling** — never store plain text. Supabase does this for you, but understand *what* it's doing.
5. **Rate limiting + abuse prevention** — for the day someone tries to spam your /api endpoints.

Best resources

- **OWASP Top 10** page itself (owasp.org).
- **Lucia auth docs** — even if you don't use Lucia, the docs explain auth concepts clearly.
- **Troy Hunt's blog** (troyhunt.com) — for cautionary tales.

06 *Design fundamentals (you have intuition; learn the names)* **NOW**

You're an artist. You have taste. The gap is **vocabulary**. Knowing the names lets you talk to designers, brief other people, and look at your own work with diagnostic eyes.

What to learn first

1. **Type scale + hierarchy** — why 11/13/15/22 px, not arbitrary sizes. Why line-height should usually be 1.4–1.6.
2. **Spacing scale** — 4 / 8 / 12 / 16 / 24 / 32 / 48. The geometric/Fibonacci ratio between sizes is what makes a layout feel "tuned" rather than thrown together.
3. **Colour theory for screens** — HSL > RGB for thinking. Saturation cascades. Why your warm-earth palette works (analogous hues + slight saturation drop on muted text).
4. **Accessibility basics** — contrast ratios (4.5:1 minimum), keyboard navigation, screen-reader semantics, focus states.
5. **Fitts's Law + touch targets** — 44×44 px minimum. Why you spent a day making the dot sliders bigger.

Best resources

- **"Refactoring UI"** — Adam Wathan + Steve Schoger. Best book for a non-designer-becoming-designer.
- **"Practical Typography"** by Matthew Butterick — practicaltypography.com.
- **WebAIM contrast checker** — bookmark it.

You don't need to learn Figma deeply. You're sketching ideas in code with my help — that's a faster loop for a builder of your kind. Use Figma when collaborating with someone else; otherwise sketch in code.

You're already doing this — using me to ship features. Most developers haven't yet figured out what works. The next 2 years, the gap between "good prompter" and "ok prompter" will be the biggest single productivity gap of any era of programming. Be in the front group.

What to learn first

1. **Specification clarity** — the more precise your ask, the better the output. *"Add a save button"* vs. *"Add a save button to MagicMaker that captures root + scale + active loop layers and POSTs to /api/notebook with category 'ideas' and tag 'magic-maker'."* — these get wildly different results.
2. **When to delegate vs. learn** — repeated patterns: delegate. Anything you'll need to debug at 3am: learn first.
3. **Reading code AI generates** — never merge what you can't read. Even quickly. *Especially* quickly.
4. **Spec-first vs. code-first** — you've started using docs/specs/ heavily. That's the right pattern: align on shape in prose, then have AI write the code.
5. **Editor integration** — Claude Code, Cursor, Continue.dev. Each has tradeoffs. Pick one and master it.

Best resources

- **Anthropic's prompting guide** — docs.anthropic.com.
- **Reading other people's spec docs** — Stripe's API guidelines, Linear's design notes. The clarity is the lesson.
- **Try every tool yourself for 1 week** — your own experience is more useful than any guide.

You're the product person on this app. There's no PM coming to save you. Most apps die because the founder builds what they think users want, not what users actually do.

What to learn first

1. **Talking to users** — open-ended questions, no leading. *"Tell me about the last time you wanted to relax with music"* not *"Would you use a meditation app?"*
2. **Watching users use the product** — Maze, Lookback, or just call your friend on FaceTime and ask them to share their screen. The single most valuable activity in product work.
3. **Jobs to be Done** framework — what is the user "hiring" your app to do for them? Not features; outcomes.
4. **The minimum lovable product** idea (bigger than minimum viable) — the smallest thing someone will love, not just tolerate.
5. **Saying no** — for every feature you ship, you commit to maintaining it for years. Half your skill as a product person is rejecting things gracefully.

Best resources

- **"The Mom Test"** by Rob Fitzpatrick — short, brilliant, about user research.
- **"Inspired"** by Marty Cagan — the canonical product book.
- **Julian Lehr's blog** (julian.digital) — designer-product-engineer thinking distilled.

The "Patagonia of social, not Meta" thesis you've written is a *business* thesis as much as a product one. There's a body of indie / lifestyle / bootstrapped business knowledge that you need before you scale, not after.

What to learn first

1. **Pricing fundamentals** — why €69/year is more sustainable than €5/month for an intentional product. When to charge upfront vs. recurring.
2. **Bootstrapping vs. raising** — what each path costs you in control, in pace, in sleep.
3. **Cash flow reading** — the difference between MRR, ARR, gross margin, LTV, CAC. A musician's bookkeeping won't survive the app's accounting needs.
4. **Customer support** — what tools (Plain, Help Scout, Intercom), what tone, what auto-response.
5. **Vendor + tool spend** — Vercel, Supabase, OpenAI, Stripe. These add up. Watch them monthly.

Best resources

- **"The Embedded Entrepreneur"** — Arvid Kahl.
- **Indie Hackers podcast** — for stories of people who've been here.
- **Lenny Rachitsky's newsletter** — covers a wider range but consistently useful.

10 *The master's at Politecnico Milano (if you go)*

OPTIONAL

Politecnico's *Communication Design* and *Digital and Interaction Design* programs are world-class. They'll give you four things this curriculum can't: **peers, structure, credentials, time.**

What to expect

1. **Studio culture** — most learning happens between students, not in lectures. Be in the studio.
2. **Italian design tradition** — Munari, Castiglioni, Sottsass. Read about them before you arrive.
3. **Critical theory you'll resist** — semiotics, post-structuralism. Push through; it changes how you see images and interfaces.
4. **The thesis project** — they'll let you make Colourmap your thesis. Do that.
5. **Internship pipeline** — Milan-based studios (Cerri, Habits, Frog, Pentagram Italy) recruit out of Politecnico. Useful network even if you stay indie.

What to NOT expect

- Being taught modern web stack technically. They're a design school. The CS rigor is in their engineering programs.
- Permission to slow down on Colourmap. The opposite — they'll want you to ship more, not less.

The school is worth it for the network and the structured time, not the technical curriculum. If you can't go, replicate it: a small group of 5 builders meeting weekly + a published portfolio + a 4-month focused thesis-style project. That's 80% of the value at 0% of the cost.

Ten-week starter plan

If you have 5 hours/week to invest in learning *outside* shipping, here's a sequence:

Week	Focus	Concrete output
1	JavaScript closures + promises (Module 01)	Read Comeau articles; rewrite one of your useEffects from memory
2	TypeScript narrowing + generics (Module 01)	Type-tighten one of your service files
3	Web Audio graph + oscillators (Module 02)	Build a one-knob synth from scratch in a sandbox
4	Sound design ADSR + filters (Module 02 + 03)	Recreate one of your existing instruments without using any existing code
5	SQL fundamentals + schema design (Module 04)	Sketch the Circle Missions schema without help; compare to mine
6	Refactoring UI book chapters 1–4 (Module 06)	Apply spacing scale + type scale to one Colourmap surface
7	The Mom Test book (Module 08)	Run 3 user interviews with friends-of-friends
8	OWASP Top 10 + Supabase auth (Module 05)	Audit one Colourmap route for security smells
9	Pricing fundamentals (Module 09)	Sketch pricing for Pro tier; A/B in friends' heads
10	Reflection + plan next 10	Update this curriculum based on what you learned

. . .

The honest closing

You don't need to be a great engineer. You need to be a great artist who knows enough engineering to build the rooms they imagine. That's a different (rarer) thing — and it's exactly what Colourmap needs at the helm.

Use AI as your engineer-in-residence. Read just enough to argue with me when I'm wrong. Keep your taste sharp. The technical depth you need is shallower than you think — but the depth in **music**, **design**, **product instinct**, and **community-building** is the lifelong work. Spend most of your learning time there.

Update this PDF every 3 months. The curriculum will look different at 5k users than at 100. Keep the file in the **Colourmap** secret catalog so it lives in the same room as the rest of the thinking.

Colourmap · From organization to clarity.

Skills curriculum v1, written 2026-04-25 · 10 modules · ~10 pages · iterative